

1 Abstract

This project studies short-term price prediction during the NASDAQ closing auction, using the Optiver “Trading at the Close” problem as a basis. The task is to predict a stock’s 60-second future price movement relative to a synthetic market index, using order-book, auction, and market-wide features. Each stock-day is treated as a 55-step time sequence, and three models are compared: a five-state Markov model, LightGBM, and an LSTM–Transformer with a causal local-and-global attention mask and a residual regression head. Contrary to the hypothesis that sequence modeling would outperform simpler alternatives, LightGBM achieved the best final-step validation MAE (5.4536) and competition-test MAE (5.0960), while the LSTM–Transformer performed comparably on the all-step metric but did not clearly surpass LightGBM. These results suggest that the engineered features—which already encode temporal information through rolling windows, EWMA, and lagged returns—capture much of what a sequence model would otherwise need to learn implicitly, and that careful feature engineering can rival architectural complexity for this task.

2 Introduction

The closing auction is an important part of the trading day. During the final minutes, buy and sell orders are matched to form the official closing price. Prices, order imbalance, and available liquidity can change quickly during this period.

This project is based on the Optiver Trading at the Close forecasting problem. The goal is to predict the future price movement of a Nasdaq-listed stock relative to the future movement of a synthetic market index.

The project treats each stock’s auction window as a time sequence. The pipeline builds features from the order book, auction prices, time, recent price changes, volatility, market-wide information, PCA, Random Matrix Theory, and market-state models.

The main neural model uses an LSTM followed by a Transformer with a causal local-and-global attention mask. Its output is passed through a residual regression head. The model is compared with a LightGBM model and a Markov state model.

3 Dataset

3.1 Dataset Description

The dataset contains historical observations from the daily ten-minute closing-auction period on the Nasdaq Stock Exchange. At each point in the auction window, the data provide information from both the auction process and the continuous order book.

Each observation is associated with a particular stock, trading day, and number of seconds elapsed since the beginning of the closing-auction window. The prediction target represents the stock’s future movement relative to a synthetic index.

The main dataset variables are summarized below.

- **stock_id**: A unique identifier for each stock. Not every stock is necessarily present in every time bucket.
- **date_id**: A sequential identifier for the trading day that is consistent across all stocks.
- **seconds_in_bucket**: The number of seconds elapsed since the beginning of the closing-auction window.
- **imbalance_size**: The dollar amount of auction interest that remains unmatched at the current reference price.
- **imbalance_buy_sell_flag**: The direction of the current auction imbalance:

$$\text{imbalance_buy_sell_flag} = \begin{cases} 1, & \text{buy-side imbalance,} \\ -1, & \text{sell-side imbalance,} \\ 0, & \text{no imbalance.} \end{cases}$$

- **reference_price**: The price selected by first maximizing the number of paired shares, then minimizing the remaining imbalance, and then minimizing the distance from the bid–ask midpoint.
- **matched_size**: The dollar amount of auction volume that can currently be matched at the reference price.
- **far_price**: The auction-only crossing price that maximizes the number of matched shares without including continuous-market orders.
- **near_price**: The crossing price obtained when both auction interest and continuous-market orders are considered.
- **bid_price** and **ask_price**: The prices of the most competitive buy and sell orders in the continuous order book.
- **bid_size** and **ask_size**: The dollar amounts available at the best bid and best ask.
- **wap**: The weighted average price made from the best bid, best ask, and their sizes.
- **target**: The stock’s 60-second future WAP movement relative to the corresponding 60-second future movement of the synthetic index. The target is measured in basis points and is available only in the training data.

3.2 Weighted Average Price

The weighted average price (WAP) combines the best bid and ask prices with the sizes available at those prices.

It can be written as

$$WAP_t = \frac{P_{\text{bid},t}Q_{\text{ask},t} + P_{\text{ask},t}Q_{\text{bid},t}}{Q_{\text{bid},t} + Q_{\text{ask},t}}.$$

Here, $P_{\text{bid},t}$ and $P_{\text{ask},t}$ are the best bid and ask prices. $Q_{\text{bid},t}$ and $Q_{\text{ask},t}$ are their sizes.

This weighting makes WAP respond to differences between bid-side and ask-side liquidity.

3.3 Prediction Target

The goal is not simply to predict whether a stock’s WAP will increase or decrease. Instead, the target measures the stock’s future movement relative to the market.

Let

$$R_{t,t+60}^{\text{stock}}$$

represent the 60-second future movement of a stock’s WAP and let

$$R_{t,t+60}^{\text{index}}$$

represent the corresponding movement of the synthetic index.

The target can be written as

$$y_t = 10^4 (R_{t,t+60}^{\text{stock}} - R_{t,t+60}^{\text{index}}).$$

The multiplication by 10^4 gives the result in basis points.

A positive target means the stock moved more positively than the synthetic index over the next 60 seconds. A negative target means the stock moved less positively than the index.

Because the target is relative to the market, information about other stocks and market-wide movement can be useful. A stock can increase in price but still have a negative target if the synthetic index increases by more.

3.4 Temporal Structure

The dataset contains observations throughout the ten-minute closing-auction period. In this project, observations are grouped into ten-second intervals.

The sequence for each stock and trading day contains

$$T = 55$$

timesteps. The input sequence is

$$X^{(i,d)} = (x_1, x_2, \dots, x_{55}),$$

where i represents the stock and d represents the trading day.
Each x_t contains the market and created features available at that time.
The target sequence is

$$Y^{(i,d)} = (y_1, y_2, \dots, y_{55}).$$

This allows the neural model to learn from every timestep instead of only the final timestep.

3.5 Training and Validation Split

Because the data are ordered by trading day, the split is also made by date. The dates are sorted from earliest to latest.

The last 81 trading days are used for validation, and all earlier days are used for training:

$$\mathcal{D}_{\text{train}} = \{d_1, \dots, d_{N-81}\}, \quad \mathcal{D}_{\text{val}} = \{d_{N-80}, \dots, d_N\}.$$

This keeps the validation period later than the training period.

The feature scaler and fitted feature models are learned from the training period and then applied to validation.

The synthetic-index weights inside the training data use an expanding out-of-fold method. Earlier dates are used to create the index feature for later training dates.

4 Evaluation

Model performance is measured using Mean Absolute Error (MAE).

For n observations,

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|,$$

where y_i is the true target and $\hat{y}_i$ is the model prediction.

Because the target is measured in basis points, MAE is also reported in basis points.

MAE is also used as the training loss for the neural model:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|.$$

Two forms of MAE are reported.

The first uses every timestep:

$$\text{MAE}_{\text{all}} = \frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T |y_{i,t} - \hat{y}_{i,t}|.$$

The second uses only the final timestep:

$$\text{MAE}_{\text{final}} = \frac{1}{N} \sum_{i=1}^N |y_{i,T} - \hat{y}_{i,T}|.$$

The all-step MAE measures performance during the full auction window. The final-step MAE measures performance at the end of the sequence.

5 Research Hypothesis

The main idea of this project is that the order of events during the closing auction may contain useful information.

Order imbalance, price gaps, liquidity, and volatility change during the auction. A sequence model can use both the current values and the way those values changed earlier in the auction.

The three main models are

$$\text{Markov State Model} \rightarrow \text{LightGBM} \rightarrow \text{LSTM} + \text{Transformer}.$$

The Markov model uses a small set of market states and their transition probabilities. LightGBM uses the created features one row at a time. The LSTM-Transformer uses the full ordered sequence.

The main hypothesis is

$$H_1 : \quad \text{MAE}_{\text{LSTM+Transformer}} < \text{MAE}_{\text{baselines}}.$$

If this happens, it suggests that using the time sequence gives useful information beyond the simpler models.

The project reports

$$\text{MAE}_{\text{all}}$$

for all timesteps and

$$\text{MAE}_{\text{final}}$$

for the final timestep.

6 Feature Engineering

The code creates several groups of features. The goal is to describe the current order book, auction prices, recent changes, market-wide movement, volatility, and market state.

6.1 Basic Order-Book and Auction Features

The basic features include the bid–ask spread,

$$\text{Spread} = P_{\text{ask}} - P_{\text{bid}},$$

and the midpoint,

$$P_{\text{mid}} = \frac{P_{\text{bid}} + P_{\text{ask}}}{2}.$$

The model also uses several size and imbalance ratios.
Book imbalance is

$$\text{Book Imbalance} = \frac{\text{Bid Size} - \text{Ask Size}}{\text{Bid Size} + \text{Ask Size} + \epsilon}.$$

The code also creates signed auction imbalance features, such as

$$\text{Signed Imbalance Size} = \text{Imbalance Size} \times \text{Buy/Sell Flag}.$$

These features combine the size and direction of the auction imbalance.

6.2 Microprice Features

The microprice uses the best bid, best ask, and their sizes:

$$P_{\text{micro}} = \frac{P_{\text{bid}}Q_{\text{ask}} + P_{\text{ask}}Q_{\text{bid}}}{Q_{\text{bid}} + Q_{\text{ask}} + \epsilon}.$$

The model also uses

$$P_{\text{micro}} - P_{\text{mid}}.$$

This shows how the size difference between the bid and ask sides moves the microprice away from the midpoint.

6.3 Far Price, Near Price, and Auction-Time Features

The code uses `far_price`, `near_price`, and `seconds_in_bucket` to create auction-specific features.

Examples include

$$P_{\text{far}} - P_{\text{near}}, \quad P_{\text{far}} - P_{\text{ref}}, \quad P_{\text{near}} - P_{\text{ref}}.$$

Differences from WAP are also used.

The code also uses relative values, such as

$$\frac{P_{\text{near}} - P_{\text{wap}}}{|P_{\text{wap}}| + \epsilon}.$$

Auction progress is

$$\text{Auction Progress} = \frac{\text{Seconds in Bucket}}{540}.$$

The fraction of time left is

$$\text{Time Left Fraction} = \frac{540 - \text{Seconds in Bucket}}{540}.$$

Time is also combined with other features.

For example,

$$\text{Auction Pressure} = \text{Near-WAP Relative Gap} \times \text{Signed Imbalance Ratio} \times \text{Auction Progress}.$$

Changes in far price and near price are also included. This allows the model to see both their current values and how they are changing during the auction.

6.4 Price Combination Features

The code creates differences and ratios between several prices:

- reference price,
- bid price,
- ask price,
- WAP,
- midpoint.

It also creates features from groups of three prices by comparing the highest, middle, and lowest values.

These features help the model measure how far different prices are from each other.

6.5 Sequence and Rolling Features

The code creates WAP returns using earlier timesteps.

It uses the one-step WAP return and additional return periods of 2, 3, and 5 timesteps.

Rolling windows of 3, 5, and 10 timesteps are also used.

These include:

- average imbalance ratio,
- average bid–ask spread,
- WAP standard deviation.

These calculations only use the current and earlier values.

6.6 EWMA and Smoothed Features

Exponentially weighted moving averages are created for:

- WAP,
- imbalance ratio,
- bid-ask spread.

Fast and slow spans are used to represent shorter-term and longer-term movement.

The code also creates a smoothed WAP using the current and earlier observations.

The difference

$$\text{WAP} - \text{Smoothed WAP}$$

shows how far the current WAP is from its recent smoothed value.

6.7 Volatility Features

Several measures of recent price movement are used.

EWMA volatility has the form

$$\sigma_t^2 = (1 - \lambda)r_t^2 + \lambda\sigma_{t-1}^2.$$

The code also uses a GARCH-style calculation,

$$\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta\sigma_{t-1}^2.$$

The values of ω , α , and β are fixed in the code.

Realized variation is calculated from recent squared returns:

$$RV_t = \sum_{j=t-k+1}^t r_j^2.$$

Bipower variation is

$$BV_t \approx \frac{\pi}{2} \sum_{j=t-k+2}^t |r_j||r_{j-1}|.$$

A jump feature is then calculated as

$$J_t = \max(RV_t - BV_t, 0).$$

The code also compares fast and slow EWMA volatility to create a volatility-regime ratio.

6.8 Outlier Feature

The code uses a trailing Hampel-style check to find WAP values that are far from their recent median.

Only the current and earlier values are used.

The output is a zero-or-one feature that tells the model whether the current WAP looks unusual compared with its recent values.

6.9 Cross-Sectional Features

Cross-sectional features compare a stock with other stocks at the same date and time.

For a feature x_i , the z-score is

$$z_i = \frac{x_i - \mu_t}{\sigma_t},$$

where μ_t and σ_t are calculated across stocks at the same timestamp.

The code creates ranks and z-scores for:

- WAP return,
- imbalance ratio,
- book imbalance.

These features show whether a stock has a high or low value compared with the rest of the market at that time.

6.10 Stock-Level Summary Features

The code calculates stock-level summary values from the training period.

These are based on variables such as:

- WAP,
- bid-ask spread,
- matched size,
- imbalance ratio,
- imbalance size,
- bid size,
- ask size.

These features give the model information about the usual values for each stock.

They are calculated from the training data and then applied to the validation and test data.

6.11 Synthetic-Index Features

The target is defined relative to a synthetic index, so the code estimates stock weights for an index-like return.

The resulting features are

Index Return

and

Stock Minus Index Return = Stock Return – Index Return.

For the training set, the index weights are created with an expanding out-of-fold method.

Earlier dates are used to create weights for later training blocks.

6.12 Random Matrix Theory Features

Random Matrix Theory is used to find common return patterns across stocks.

A stock correlation matrix is created and written as

$$C = V\Lambda V^T.$$

The code keeps the first three factor directions.

For each stock, the code creates:

- factor loadings,
- factor returns,
- stock return minus each factor return.

The Marchenko–Pastur upper value used in the code is

$$\lambda_+ = (1 + \sqrt{q})^2, \quad q = \frac{N}{T}.$$

The code also stores the fraction connected to the top eigenvalue and the number of eigenvalues above this value.

The main goal of these features is to separate common market movement from stock-specific movement.

6.13 Soft Market-State Features

A three-state soft market-state filter is built from stock return minus the reconstructed index return.

Instead of assigning only one state, the model gives three probabilities:

$$P(S_t = 1), \quad P(S_t = 2), \quad P(S_t = 3).$$

These probabilities are used as features.

The code also calculates an HMM-style expected target, but this value is removed from the final feature list before model training.

Therefore, the target-based HMM estimate is not directly passed to LightGBM or the neural model.

6.14 PCA Features

PCA is fitted using five variables:

- imbalance ratio,
- book imbalance,
- bid-ask spread,
- size ratio,
- matched-imbalance ratio.

The first three PCA components are added as features.

These components give the model three smaller combinations of the related order-book variables.

6.15 Markov State Model

A separate five-state Markov model is built from stock return minus index return.

If S_t is the state at time t , the transition matrix is

$$P_{ij} = P(S_t = j \mid S_{t-1} = i).$$

Each state also has a mean target value.

The Markov prediction is

$$\hat{y}_t = \sum_j P_{ij} \mu_j.$$

This prediction is used for the Markov baseline.

It is not added to the main neural-model feature list.

6.16 Feature Selection

Before the final LightGBM and neural model are trained, the code runs a LightGBM feature-selection step.

Each feature receives an importance value.

The importance values are changed into fractions, and a feature is kept when

$$\text{Importance Fraction} \geq 0.004.$$

This removes features with very low importance and makes the final feature set smaller.

7 Model Architecture

The main neural model has three main parts:

Input Features $\longrightarrow$ LSTM $\longrightarrow$ Causal Local+Global Transformer $\longrightarrow$ Residual Regression Head.

For each stock and date, the input is

$$X = (x_1, x_2, \dots, x_T),$$

with

$$T = 55.$$

The model produces one prediction at every timestep:

$$\hat{Y} = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_T).$$

This allows the same model to be evaluated during the full auction and at the final timestep.

7.1 LSTM Representation

The first part of the model is a two-layer LSTM.

The hidden size is

$$d_h = 96.$$

At timestep t ,

$$h_t = \text{LSTM}(x_t, h_{t-1}).$$

The LSTM carries information from earlier timesteps through the sequence.

7.2 Causal Local-and-Global Transformer

The LSTM outputs are passed into a one-layer Transformer encoder with four attention heads.

The Transformer uses a causal mask. This means that the prediction at time t cannot use information from a future timestep.

The model can attend to:

- the most recent 15 timesteps,
- earlier global positions spaced every 2 timesteps.

Each timestep represents 10 seconds, so the global positions are spaced every 20 seconds.

If q is the current position and k is another position, the first condition is

$$k \leq q.$$

This prevents the model from using future information.
For the local part,

$$k \geq q - 14.$$

This means the current position can use the latest 15 timesteps.

Positions farther back can still be used when they are one of the global positions.

This gives the model detailed information from the recent past while still allowing it to use information from earlier parts of the auction.

7.3 Residual Regression Head

The final prediction head is not only one linear layer.

First, layer normalization is applied.

The model then makes two predictions and adds them together:

$$\hat{y}_t = \hat{y}_t^{\text{linear}} + \hat{y}_t^{\text{residual}}.$$

The first part is a linear prediction:

$$\hat{y}_t^{\text{linear}} = W_{\text{lin}} h_t + b_{\text{lin}}.$$

The second part is a small neural network:

$$h_t \longrightarrow \text{Linear} \longrightarrow \text{GELU} \longrightarrow \text{Dropout} \longrightarrow \text{Linear}.$$

The hidden size is reduced from 96 to 48 inside this part.

The idea is simple. The linear part makes a basic prediction, and the small neural network learns a correction to that prediction.

8 Model Training

The neural model is trained using Mean Absolute Error with PyTorch's L_1 loss:

$$\mathcal{L} = \frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T |y_{i,t} - \hat{y}_{i,t}|.$$

Every timestep contributes to the training loss.

The optimizer is Adam.

The peak learning rate is

$$\eta_{\max} = 10^{-4}.$$

A OneCycle learning-rate schedule is used with cosine annealing.

The first 30% of the schedule is used for the increasing part of the learning-rate cycle.

The main training settings are

$$\text{batch size} = 64,$$

$$\text{weight decay} = 10^{-4},$$

and

$$\text{dropout} = 0.3.$$

Gradient clipping limits the gradient norm:

$$\|\nabla_{\theta} \mathcal{L}\| \leq 1.$$

Training can run for up to 40 epochs.

Early stopping is used if the all-step validation MAE does not improve for 15 epochs.

The saved model is the model from the epoch with the lowest all-step validation MAE.

Final-step MAE is then calculated separately using that saved model.

9 Baseline and Comparison Models

The final code compares three models:

1. a five-state Markov model,
2. a LightGBM model,
3. the LSTM-Transformer model with the residual regression head.

All three models are evaluated using both all-step MAE and final-step MAE.

9.1 Markov State Baseline

The Markov model divides stock-minus-index return into five states.

Its transition matrix is

$$P_{ij} = P(S_t = j \mid S_{t-1} = i).$$

Each state has a mean target value μ_j .

Given the previous state, the prediction is

$$\hat{y}_t = \sum_j P_{ij} \mu_j.$$

For the first timestep, there is no previous state. In this case, the code uses the overall mean target from the training data.

The Markov model provides a simple time-based baseline.

9.2 LightGBM Baseline

LightGBM is the main row-based machine-learning baseline.

Each timestep is treated as one feature vector:

$$x_t \longrightarrow \hat{y}_t.$$

The model is trained using rows from all timesteps.

Its main settings are

number of trees ≤ 3000 ,

learning rate = 0.02,

number of leaves = 63,

subsample = 0.8,

and

column subsample = 0.8.

The objective is MAE.

Early stopping uses the validation period and stops after 100 rounds without improvement.

9.3 LSTM–Transformer Model

The neural model uses

Input $\longrightarrow$ 2-Layer LSTM $\longrightarrow$ 1-Layer Transformer $\longrightarrow$ Residual Regression Head.

Unlike LightGBM, the neural model keeps the order of the 55 auction timesteps.

The Transformer is causal and uses the local-and-global attention mask described earlier.

10 Model Evaluation Framework

The code reports results in two main ways.

10.1 All-Step MAE

All predictions in every sequence are included:

$$\text{MAE}_{\text{all}} = \frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T |y_{i,t} - \hat{y}_{i,t}|.$$

This measures model performance over the whole auction window.

10.2 Final-Step MAE

Only the final prediction from each stock-date sequence is included:

$$\text{MAE}_{\text{final}} = \frac{1}{N} \sum_{i=1}^N |y_{i,T} - \hat{y}_{i,T}|.$$

This measures model performance at the final point in the sequence.

10.3 Prediction Error Across the Auction Window

The code also calculates MAE separately at every timestep:

$$E_t = \frac{1}{N} \sum_{i=1}^N |y_{i,t} - \hat{y}_{i,t}|.$$

The sequence

$$(E_1, E_2, \dots, E_T)$$

is plotted against seconds into the auction.

This graph shows how prediction error changes from the beginning of the auction to the final timestep.

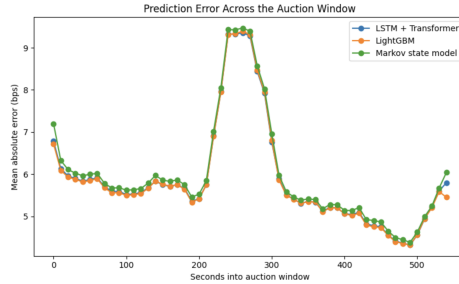


Figure 1: Prediction Error Across the auction Window

10.4 Competition Test Check

The code also calculates MAE for the test rows that have revealed targets.

The same fitted scaler, feature models, Markov model, LightGBM model, and neural model are applied to the test data.

The supplied `test.csv` overlaps with the end of `train.csv`. Because of this, the competition-test MAE should be treated as a competition or replay check rather than a fully independent test score.

10.5 Overall Comparison

The final comparison is

Markov $\rightarrow$ LightGBM $\rightarrow$ LSTM + Transformer

using both all-step and final-step MAE.

The Markov model tests a simple state-transition method.

LightGBM tests whether the created features can predict the target without directly modeling the full sequence.

The LSTM-Transformer tests whether keeping the full time order improves the prediction.

```
=====
FINAL STEP COMPARISON (LAST-81-DAY VALIDATION vs COMPETITION TEST; MAE in $ps, lower is better)
=====
Model Category Validation MAE Competition Test MAE
LightGBM Baseline 5.4536 5.0960
LSTM + Transformer Neural 5.8026 5.2561
Markov state model Baseline 6.9131 5.3838
=====
```

Figure 2: Final Step Comparison

```
=====
ALL STEPS COMPARISON (LAST-81-DAY VALIDATION vs COMPETITION TEST; MAE in $ps, lower is better)
=====
Model Category Validation MAE Competition Test MAE
LSTM + Transformer Neural 5.8984 5.4212
LightGBM Baseline 5.8734 5.4215
Markov state model Baseline 5.9947 5.4678
=====
```

Figure 3: All Steps Comparison

11 Conclusion

This project tested several methods for predicting short-term stock price movement during the NASDAQ closing auction. The final comparison included a Markov state model, LightGBM, and an LSTM-Transformer sequence model.

For the final timestep, LightGBM achieved the best validation MAE of

5.4536

and the best competition test MAE of

5.0960.

The LSTM-Transformer reached a validation MAE of

5.8026

and a competition test MAE of

5.2561,

while the Markov state model had the highest errors, with a validation MAE of

6.0431

and a competition test MAE of

5.3838.

For predictions across all timesteps, LightGBM and the LSTM–Transformer performed very similarly. LightGBM had a validation MAE of 5.8734, while the LSTM–Transformer had a validation MAE of 5.8884. On the competition test, the LSTM–Transformer had an MAE of 5.4212 and LightGBM had an MAE of 5.4215. The difference between these two results is very small.

These results do not support the original hypothesis that the sequence model would clearly outperform the simpler models. LightGBM performed better at the final timestep and was almost equal to the LSTM–Transformer across the full auction window. This suggests that the engineered features already contain much of the useful information needed for prediction. The LSTM and Transformer were able to use the time sequence, but this added complexity did not produce a clear improvement over LightGBM.

The Markov model performed worse than both machine-learning models. This suggests that dividing the market into a small number of states is too simple to fully describe the behavior of the closing auction. However, its results were still reasonably close to the other models, showing that changes between market states contain some useful information.

The prediction-error graph also shows that all three models follow almost the same error pattern during the auction. Error decreases during several parts of the window but rises sharply around 240–280 seconds. After this period, error falls again and reaches its lowest level near the end of the auction before increasing during the final timesteps. Because all three models show almost the same pattern, this increase is likely related to changes in the data during that part of the auction rather than to one specific model.

For reference, the results reported by the competition winner were also considered. Their models included CatBoost, GRU, Transformer, GRU + Transformer, and CatBoost + GRU + Transformer combinations. Their reported validation MAE values were around 5.81–5.86, while their test MAE values were around 5.44–5.47. These numbers are included only as a reference point and were not produced by the models implemented in this project.

Comparing the results shows that the models in this project reach a similar general MAE range, although the datasets, validation setup, preprocessing, and evaluation details may not be exactly the same. Because of this, the competition-winner results should not be treated as a direct one-to-one comparison.

Overall, the project shows that careful feature engineering and validation can be at least as important as using a more complex neural network. LightGBM provided the strongest final-step results, while the LSTM-Transformer remained competitive across the full sequence. Future work could focus on understanding the periods where prediction error increases, improving auction-price and market-wide features, and testing whether different sequence architectures can make better use of changes that occur during the auction.

12 References

1. Zivot, E. and Wang, J. Modeling Financial Time Series with S-PLUS. 2nd ed., Springer, 2006.
2. Optiver. “Optiver – Trading at the Close.” Kaggle Competition. <https://www.kaggle.com/competitions/optiver-trading-at-the-close>
3. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree.” Advances in Neural Information Processing Systems, 2017.
4. Hochreiter, S. and Schmidhuber, J. “Long Short-Term Memory.” Neural Computation, 9(8), 1735–1780, 1997.
5. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. “Attention Is All You Need.” Advances in Neural Information Processing Systems, 2017.
6. Engle, R. F. “Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation.” Econometrica, 50(4), 987–1007, 1982.
7. Marchenko, V. A. and Pastur, L. A. “Distribution of Eigenvalues for Some Sets of Random Matrices.” Mathematics of the USSR-Sbornik, 1(4), 457–483, 1967.
8. Jolliffe, I. T. Principal Component Analysis. Springer.
9. Rabiner, L. R. “A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition.” Proceedings of the IEEE, 77(2), 257–286, 1989.
10. Bollerslev, T. “Generalized Autoregressive Conditional Heteroskedasticity.” Journal of Econometrics, 31(3), 307–327, 1986.